```
// ---------- Storage ----------
const STORAGE_KEY = 'session_presets_v1';

const DEFAULT_PRESETS = [
  { id: 'p45-5',  focus: 45, brk: 5,  label: '45-5' },
  { id: 'p45-7',  focus: 45, brk: 7,  label: '45-7' },
  { id: 'p45-10', focus: 45, brk: 10, label: '45-10' },
  { id: 'p30-5',  focus: 30, brk: 5,  label: '30-5' },
  { id: 'p30-7',  focus: 30, brk: 7,  label: '30-7' },
  { id: 'p30-10', focus: 30, brk: 10, label: '30-10' },
];

function loadPresets() {
  try {
    const raw = localStorage.getItem(STORAGE_KEY);
    if (!raw) return DEFAULT_PRESETS.slice();
    const parsed = JSON.parse(raw);
    if (Array.isArray(parsed) && parsed.length) return parsed;
    return DEFAULT_PRESETS.slice();
  } catch (e) {
    return DEFAULT_PRESETS.slice();
  }
}

function savePresets(presets) {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(presets));
}

let presets = loadPresets();
let activePresetId = presets[0].id;
let selectedSets = 3;

// ---------- Timer state ----------
// phase: 'idle' | 'focus' | 'break' | 'done'
let timer = {
  phase: 'idle',
  currentSet: 1,
  totalSets: 3,
  phaseEndAt: null,   // epoch ms when current phase should end
  remainingMsAtPause: null,
  paused: false,
};

let tickHandle = null;
```

```
let wakeLock = null;

// ---------- DOM refs ----------
const tabbar = document.getElementById('tabbar');
const setupView = document.getElementById('setupView');
const runView = document.getElementById('runView');
const runControls = document.getElementById('runControls');
const presetLabel = document.getElementById('presetLabel');
const presetDetail = document.getElementById('presetDetail');
const setPicker = document.getElementById('setPicker');
const startBtn = document.getElementById('startBtn');
const pauseBtn = document.getElementById('pauseBtn');
const stopBtn = document.getElementById('stopBtn');
const clockDisplay = document.getElementById('clockDisplay');
const runEyebrow = document.getElementById('runEyebrow');
const runSubline = document.getElementById('runSubline');
const ringFg = document.getElementById('ringFg');
const body = document.body;

const RING_CIRC = 2 * Math.PI * 92;
ringFg.style.strokeDasharray = `${RING_CIRC}`;
ringFg.style.strokeDashoffset = `${RING_CIRC}`;

// ---------- Rendering: tabs ----------
function renderTabs() {
  tabbar.innerHTML = '';
  presets.forEach(p => {
    const btn = document.createElement('button');
    btn.className = 'tab' + (p.id === activePresetId ? ' active' : '');
    btn.textContent = p.label;
    btn.addEventListener('click', () => {
      if (timer.phase !== 'idle') return; // don't allow tab switch mid-run
      activePresetId = p.id;
      renderTabs();
      renderSetup();
    });
    btn.addEventListener('contextmenu', (e) => {
      e.preventDefault();
      openEditor(p.id);
    });
    let pressTimer;
    btn.addEventListener('touchstart', () => {
      pressTimer = setTimeout(() => openEditor(p.id), 550);
    });
    btn.addEventListener('touchend', () => clearTimeout(pressTimer));
    tabbar.appendChild(btn);
  });
```

```
  const addBtn = document.createElement('button');
  addBtn.className = 'tab add-tab';
  addBtn.textContent = '＋';
  addBtn.addEventListener('click', () => openEditor(null));
  tabbar.appendChild(addBtn);
}

function getActivePreset() {
  return presets.find(p => p.id === activePresetId) || presets[0];
}

// ---------- Rendering: setup view ----------
function renderSetup() {
  const p = getActivePreset();
  presetLabel.textContent = p.label;
  presetDetail.textContent = `集中 ${p.focus}分 ・ 休憩 ${p.brk}分`;

  setPicker.innerHTML = '';
  for (let i = 1; i <= 8; i++) {
    const b = document.createElement('button');
    b.className = 'set-btn' + (i === selectedSets ? ' selected' : '');
    b.textContent = i;
    b.addEventListener('click', () => {
      selectedSets = i;
      renderSetup();
    });
    setPicker.appendChild(b);
  }
}

// ---------- Timer control ----------
function startSession() {
  const p = getActivePreset();
  timer.phase = 'focus';
  timer.currentSet = 1;
  timer.totalSets = selectedSets;
  timer.paused = false;
  timer.phaseEndAt = Date.now() + p.focus * 60 * 1000;
  timer.phaseDurationMs = p.focus * 60 * 1000;

  setupView.classList.add('hidden');
  runView.classList.remove('hidden');
  runControls.classList.remove('hidden');
  pauseBtn.textContent = '一時停止';

  requestWakeLock();
  startTicking();
```

```
  render();
}

function advancePhase() {
  const p = getActivePreset();
  if (timer.phase === 'focus') {
    timer.phase = 'break';
    timer.phaseDurationMs = p.brk * 60 * 1000;
    timer.phaseEndAt = Date.now() + timer.phaseDurationMs;
    playChime('break');
  } else if (timer.phase === 'break') {
    if (timer.currentSet >= timer.totalSets) {
      finishSession();
      return;
    }
    timer.currentSet += 1;
    timer.phase = 'focus';
    timer.phaseDurationMs = p.focus * 60 * 1000;
    timer.phaseEndAt = Date.now() + timer.phaseDurationMs;
    playChime('focus');
  }
  render();
}

function finishSession() {
  playChime('done');
  timer.phase = 'idle';
  stopTicking();
  releaseWakeLock();
  runView.classList.add('hidden');
  runControls.classList.add('hidden');
  setupView.classList.remove('hidden');
  body.classList.remove('mode-focus', 'mode-break');
  renderSetup();
}

function stopSessionManually() {
  playChime(null);
  timer.phase = 'idle';
  stopTicking();
  releaseWakeLock();
  runView.classList.add('hidden');
  runControls.classList.add('hidden');
  setupView.classList.remove('hidden');
  body.classList.remove('mode-focus', 'mode-break');
  renderSetup();
}
```

```
function togglePause() {
  if (timer.phase === 'idle') return;
  if (!timer.paused) {
    timer.paused = true;
    timer.remainingMsAtPause = timer.phaseEndAt - Date.now();
    pauseBtn.textContent = '再開する';
    stopTicking();
  } else {
    timer.paused = false;
    timer.phaseEndAt = Date.now() + timer.remainingMsAtPause;
    pauseBtn.textContent = '一時停止';
    startTicking();
  }
}

function startTicking() {
  stopTicking();
  tickHandle = setInterval(tick, 250);
  tick();
}
function stopTicking() {
  if (tickHandle) clearInterval(tickHandle);
  tickHandle = null;
}

function tick() {
  if (timer.phase === 'idle' || timer.paused) return;
  const remaining = timer.phaseEndAt - Date.now();
  if (remaining <= 0) {
    advancePhase();
    return;
  }
  render(remaining);
}

// ---------- Rendering: run view ----------
function render(remainingOverride) {
  if (timer.phase === 'idle') return;
  const remaining = remainingOverride !== undefined
    ? remainingOverride
    : Math.max(0, timer.phaseEndAt - Date.now());

  const totalSec = Math.ceil(remaining / 1000);
  const mm = Math.floor(totalSec / 60).toString().padStart(2, '0');
  const ss = (totalSec % 60).toString().padStart(2, '0');
  clockDisplay.textContent = `${mm}:${ss}`;
```

```
  const isFocus = timer.phase === 'focus';
  runEyebrow.textContent = isFocus ? '集中' : '休憩';
  runEyebrow.className = 'eyebrow ' + (isFocus ? 'state-focus' : 'state-break');
  runSubline.textContent = `セット ${timer.currentSet} / ${timer.totalSets}`;

  body.classList.toggle('mode-focus', isFocus);
  body.classList.toggle('mode-break', !isFocus);

  const progress = 1 - (remaining / timer.phaseDurationMs);
  const offset = RING_CIRC * (1 - Math.max(0, Math.min(1, progress)));
  ringFg.style.strokeDashoffset = `${offset}`;
}

// ---------- Recompute on resume (tab foregrounded after being backgrounded) ---
document.addEventListener('visibilitychange', () => {
  if (document.visibilityState === 'visible' && timer.phase !== 'idle' && !timer.
    // catch up: if we overshot the phase end while backgrounded, advance phases
    let guard = 0;
    while (timer.phaseEndAt <= Date.now() && timer.phase !== 'idle' && guard < 50
      advancePhase();
      guard++;
    }
    startTicking();
    requestWakeLock();
  }
});

// ---------- Wake Lock ----------
async function requestWakeLock() {
  try {
    if ('wakeLock' in navigator) {
      wakeLock = await navigator.wakeLock.request('screen');
    }
  } catch (e) { /* ignore */ }
}
function releaseWakeLock() {
  if (wakeLock) {
    wakeLock.release().catch(() => {});
    wakeLock = null;
  }
}

// ---------- Audio chime ----------
let audioCtx = null;
function playChime(kind) {
  if (!kind) return;
```

```
  try {
    audioCtx = audioCtx || new (window.AudioContext || window.webkitAudioContext)
    const now = audioCtx.currentTime;
    const notes = kind === 'focus' ? [660, 880] : kind === 'break' ? [523, 440] :
    notes.forEach((freq, i) => {
      const osc = audioCtx.createOscillator();
      const gain = audioCtx.createGain();
      osc.type = 'sine';
      osc.frequency.value = freq;
      gain.gain.setValueAtTime(0.0001, now + i * 0.28);
      gain.gain.exponentialRampToValueAtTime(0.35, now + i * 0.28 + 0.03);
      gain.gain.exponentialRampToValueAtTime(0.0001, now + i * 0.28 + 0.32);
      osc.connect(gain).connect(audioCtx.destination);
      osc.start(now + i * 0.28);
      osc.stop(now + i * 0.28 + 0.35);
    });
    if (navigator.vibrate) navigator.vibrate(kind === 'done' ? [120, 60, 120, 60,
  } catch (e) { /* ignore */ }
}

// ---------- Editor modal ----------
function openEditor(presetId) {
  const editing = presetId ? presets.find(p => p.id === presetId) : null;

  const backdrop = document.createElement('div');
  backdrop.className = 'modal-backdrop';
  backdrop.innerHTML = `
    <div class="modal">
      <button class="close-x" id="modalClose">×</button>
      <h2>${editing ? 'プリセットを編集' : '新しいプリセット'}</h2>
      <div class="field-row">
        <div class="field">
          <label>集中（分）</label>
          <input type="number" inputmode="numeric" id="focusInput" value="${editi
        </div>
        <div class="field">
          <label>休憩（分）</label>
          <input type="number" inputmode="numeric" id="breakInput" value="${editi
        </div>
      </div>
      <div class="modal-actions">
        ${editing ? '<button class="modal-btn delete" id="deleteBtn">削除</button>
        <button class="modal-btn primary" id="saveBtn">保存</button>
      </div>
    </div>
  `;
  document.body.appendChild(backdrop);
```

```
  document.getElementById('modalClose').addEventListener('click', () => backdrop.
  backdrop.addEventListener('click', (e) => { if (e.target === backdrop) backdrop

  document.getElementById('saveBtn').addEventListener('click', () => {
    const focus = parseInt(document.getElementById('focusInput').value, 10);
    const brk = parseInt(document.getElementById('breakInput').value, 10);
    if (!focus || !brk || focus < 1 || brk < 1) return;
    const label = `${focus}-${brk}`;

    if (editing) {
      editing.focus = focus;
      editing.brk = brk;
      editing.label = label;
    } else {
      const newPreset = { id: 'p' + Date.now(), focus, brk, label };
      presets.push(newPreset);
      activePresetId = newPreset.id;
    }
    savePresets(presets);
    renderTabs();
    renderSetup();
    backdrop.remove();
  });

  const deleteBtn = document.getElementById('deleteBtn');
  if (deleteBtn) {
    deleteBtn.addEventListener('click', () => {
      if (presets.length <= 1) { backdrop.remove(); return; }
      presets = presets.filter(p => p.id !== presetId);
      if (activePresetId === presetId) activePresetId = presets[0].id;
      savePresets(presets);
      renderTabs();
      renderSetup();
      backdrop.remove();
    });
  }
}

// ---------- Wire up buttons ----------
startBtn.addEventListener('click', startSession);
pauseBtn.addEventListener('click', togglePause);
stopBtn.addEventListener('click', () => {
  if (confirm('セッションを終了しますか？')) stopSessionManually();
});

// ---------- Init ----------
```

```
renderTabs();
renderSetup();

// ---------- Service worker (for home screen / offline) ----------
if ('serviceWorker' in navigator) {
  window.addEventListener('load', () => {
    navigator.serviceWorker.register('sw.js').catch(() => {});
  });
}
```